

CS241 Coursework

Archie Harrodine
2127935

1 Introduction

This coursework was conveniently broken down into four sections which I will explain each of my design choices. This saw much use of lab 3, using much of their thread-pooling code. My other sources of information are found within the bibliography, that primarily aided me on analysis.

2 Packet analysis

There are four areas to focus on: SYN packets, SYN uniqueness, ARP responses, blacklisted URLs. For SYN packet checking, when making it to the TCP header, that is where I implemented a check of the flags, making sure the SYN flag was 1, and the rest 0.

For finding the number of unique SYN IP addresses, I made use of an Array struct, modelled after dynamically growing arrays found online, and how they are implemented within Java. Ideally it would have been better to use a hashset, however, those are far more complex to implement into C, and so the dynamically growing array was chosen. There was then two choices of adding all IPs to the array and then at the very end counting up all of the values, or to only add IPs that were not in the array already. Preferring the hashset style I chose the latter, as it was easier to design and implement. They would be equal in time complexity $O(n^2)$, but when implementing threading, the option I chose is slower, as the mutex lock is taken up during the check of the array, which means other threads would not be able to access it for a time.

For ARP responses, it was as simple as checking if it were an ARP packet, and then proceeding to check if the ARP operation was set to ARPOP_REPLY, which if it were we could conclude it was an ARP packet and increment our counter.

Finally for the blacklisted websites, it was a simple case of using the strstr() function, to check if the given domains of google and facebook were a substring of the text inside of the TCP packet payload. If one was there, it would increment the respective counter.

3 Threading

There were several routes that could have been taken in regard to threading. One of these suggested is the one thread per packet approach. This may have a lot of limits due to the nature of the task. 'One thread per X is a threading particularly good at solving problems with a distinct known number of inputs, or at least a known number of commands to perform and check. One way we could approach this for this task would be to distinguish between thread types, and have a thread to handle each check that needs to be made. For example we may create a thread per packet, but the amount of packets entering would not be worth the time taken to make each thread.

The alternative method we could use, would be threadpooling. This is the process in which we have a number of threads created that wait for an input to come to them. We have a queue that will hold and store the packets coming in, and when there is a free thread, we will broadcast them a signal to take a packet from the queue to analyse the next packet to be dequeued. This is effective in that it enables for a threads to constantly manage a large stream of data coming in at once. This comes with

a limitation that if the rate of packets is slow, we may have wasted resources in threads that are not working, in other words 'starved'.

To ensure safety in our value trackers being maintained between threads, we need to include some form of locking. This can be done with either mutex locks or read-write locks. We decided to pursue the mutex lock route for two reasons. Firstly, the CS241 lab resources used them, meaning that we had a direct frame of references for how to implement them, but additionally, the increased complexity of read-write locks aren't necessary. Read-write locks have three states: locked for reading, locked for writing, and unlocked. This allows multiple threads to read from a shared resource simultaneously, but only one thread can write to the resource at a time. This extra state of reading writing is not useful however, as we do not read very often from a shared resource; but we do write and change the value tracker with potentially each and every packet, and hence it makes more sense to maintain the simple lock and unlock logic.

4 Testing

The main area to test once having tested all of the conditions set, using the given commands and built in scripts for SYN, ARP and blacklisted URLs, we finally began testing some more of the weird cases. Firstly for SYN, we tested if it would only gather unique IPs. When using the command given it is a random source entry, however if we remove that flag, it defaults to all of them being the same packet IP, so the answer would give 100 SYN packets, and 1 unique IP, which it did. The next tests were stress tests performed on the SYN packets, which was changing the command to send upwards of 10000 packets, which a non threaded system would be unable to easily handle. This was additionally done for ARP packets, both returning their expected o(Note: I used the lab 3 queue).utputs. The strangeness that is to be considered is with regards to blacklisted URLs. The specification wants us to output the IPs when found, which is bad to do, as print is not thread safe. However, since the specification asked for it, my assumption is that there will not be a test that requires two or more threads to handle a blacklisted URL specifically; which if multiple threads write to console, then we will find the text/output becomes garbled. There may be a solution to this by mutex locking it, but I'm close to the deadline as is.

5 Bibliography

"Address Resolution Protocol (ARP) Parameters." Wwww.iana.org, www.iana.org/assignments/arp-parameters/arp-parameters.xhtml. Accessed 18 Nov. 2022.

Appaji. "Scaler Topics." Wwww.scaler.com, 8 Feb. 2022, www.scaler.com/topics/c/dynamic-array-in-c/.

Balkania. "C Dynamically Growing Array." Stack Overflow, 21 Aug. 2010, stackoverflow.com/questions/3536153/c-dynamically-growing-array.

"C - Bit Fields - Tutorialspoint." Wwww.tutorialspoint.com, www.tutorialspoint.com/cprogramming/c_bit_fields.htm.

"Check Substring Exists in a String in C." Stack Overflow, stackoverflow.com/questions/12784766/check-substring-exists-in-a-string-in-c. Accessed 18 Nov. 2022.

"Ethernet.h." Sites.uclouvain.be, sites.uclouvain.be/SystInfo/usr/include/net/ethernet.h.html. Accessed 17 Nov. 2022.

"EtherType." Wikipedia, 2 Oct. 2020, en.wikipedia.org/wiki/EtherType.

"How Can I Get the Payload of a Packet from a PCAP File Using c Program?" Quora, 2022, www.quora.com/How-can-I-get-the-payload-of-a-packet-from-a-PCAP-file-using-C-program. Accessed 18 Nov. 2022.

"How Do I Catch a Ctrl+c Event in C++?" Wwww.tutorialspoint.com, www.tutorialspoint.com/how-do-i-catch-a-ctrlplusc-event-in-cplusplus#:~:text=The%20CTRL%20%2B%20C%20is%20one. Accessed 18 Nov. 2022.

“How to Access Members for Struct in Struct - C.” Stack Overflow, stackoverflow.com/questions/9317024/how-to-access-members-for-struct-in-struct-c. Accessed 17 Nov. 2022.

“If_{arp.h}.” *Sites.uclouvain.be*, . Accessed 18 Nov. 2022.

“If_{ether.h}.” *Sites.uclouvain.be*, . Accessed 17 Nov. 2022.

INTERNET PROTO DARPA INTERNET PROGRAM PROTOCOL SPECIFICATION. datatracker.ietf.org/doc/html/rfc791#page-11.

“Ntohs() — Translate an Unsigned Short Integer into Host Byte Order.” *Www.ibm.com*, 3 Mar. 2021, www.ibm.com/docs/en/zos/2.3.0?topic=lf-ntohs-translate-unsigned-short-integer-into-host-byte-order. Accessed 16 Nov. 2022.

Postel, J. “Assigned Numbers.” *Www.rfc-Editor.org*, 1 Sept. 1981, www.rfc-editor.org/rfc/rfc790, 10.17487/RFC0790. Accessed 17 Nov. 2022.

“Signals - Catch Ctrl-C in C.” Stack Overflow, stackoverflow.com/questions/4217037/catch-ctrl-c-in-c. Accessed 17 Nov. 2022.

“Simple c Example of Doing an HTTP POST and Consuming the Response.” Stack Overflow, stackoverflow.com/questions/22077802/simple-c-example-of-doing-an-http-post-and-consuming-the-response. Accessed 18 Nov. 2022.

“Sockets - IPPROTO_IPvsIPPROTO_TCP/IPPROTO_UDP.” *StackOverflow*, . Accessed 17 Nov. 2022.

“Sockets - Why Are There Differences between ETH_PIPandETH_PALLWhen.DoingaRead().” *StackOverflow*, . Accessed 16 Nov. 2022.

Stratus, Leo. “Netinet.” GitHub, 1 Oct. 2022, github.com/leostratus/netinet/blob/master/ip.h. Accessed 17 Nov. 2022.

“String - c - the %X Format Specifier.” Stack Overflow, stackoverflow.com/questions/15108932/c-the-x-format-specifier.

“String - Integer to IP Address - C.” Stack Overflow, stackoverflow.com/questions/1680365/integer-to-ip-address-c. Accessed 18 Nov. 2022.

“Strstr() in C/C++.” *GeeksforGeeks*, 11 Oct. 2017, www.geeksforgeeks.org/strstr-in-cpp/#:~:text=In%20C%2B%2C%20std%3A%3A. Accessed 18 Nov. 2022.

“Struct Sockaddr_n, StructIn_aaddr.” *Www.gta.ufrj.br*, . . “Tcp.h.” *Sites.uclouvain.be*, sites.uclouvain.be/SystInfo/usr/include/netinet/tcp.h.html. Accessed 17 Nov. 2022.

Wikipedia Contributors. “List of IP Protocol Numbers.” Wikipedia, Wikimedia Foundation, 27 Aug. 2019, en.wikipedia.org/wiki/List_of_IP_protocol_numbers.

—. “List of TCP and UDP Port Numbers.” Wikipedia, Wikimedia Foundation, 4 June 2019, en.wikipedia.org/wiki/List_of_TCP_and_UDP_port_numbers.

“WinPcap: Pcap_pkthdrStructReference.” *Www.winpcap.org*, . Accessed 16 Nov. 2022.

Nmap.org, 2022, svn.nmap.org/nmap/struct_ip.h. Accessed 17 Nov. 2022.